

NOMBRES: Sol Ailen Kalapuj, Fiorella Mosca

NÚMEROS DE MATRÍCULA: 154106, 154108

CARRERA: Tecnicatura en Programación de Sistemas

MATERIA: Desarrollo de Aplicaciones Móviles II

EXAMEN PARCIAL 1

Análisis Arquitectónico y Desarrollo de «DevPosts Reader»

Repositorio:	2
Hash del commit de la entrega:	2
.Zip de la entrega:	2
1. MVVM en acción	2
a. El Modelo (Model)	2
b. La Vista (View)	3
c. El ViewModel (ViewModel)	3
2. Data Binding y Comandos	4
a. Verificación del BindingContext	4
b. Coincidencia nominal del enlace y Source Generators	4
d. Bloqueo del hilo de interfaz gráfica (UI Thread)	5
3. Uso del CommunityToolkit.MVVM	5
a. Ventajas constatadas en nuestro proyecto	5
b. Consideraciones y desventajas evaluadas	6
4. Consumo de APIs y Estrategia de Manejo de Errores	6
a. Estrategia de Diseño: Patrón Envoltorio (Result Pattern)	6
b. Diferenciación técnica de fallos en PostService	7
c. Justificación de la solución	8
5. Navegación en MAUI y Escalabilidad Estructural	8
a. Resolución de la Navegación en el Proyecto	8
b. Alternativa ante un proyecto con múltiples secciones	9
6. Justificación de esta elección frente a NavigationPage o TabbedPage	9

Repositorio:

<https://github.com/fioremos/UCES-DAMII-DevPosts-KalapujMosca>

Hash del commit de la entrega:

86758afa2118abd7d6a31a34f5ae3480a3db1d4f

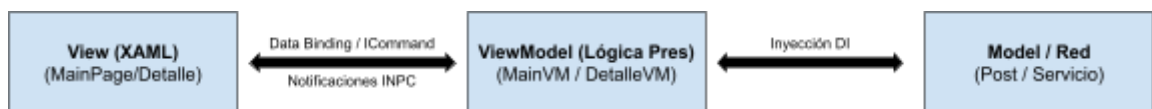
<https://github.com/fioremos/UCES-DAMII-DevPosts-KalapujMosca/tree/86758afa2118abd7d6a31a34f5ae3480a3db1d4f>

.Zip de la entrega:

<https://drive.google.com/file/d/1HA5keuvjtyq8xaj905aNZDSndu1rwMCu/view?usp=sharing>

1. MVVM en acción

En la solución DevPosts Reader, el patrón Model-View-ViewModel (MVVM) se implementó con un criterio estricto de separación de responsabilidades (SoC), buscando desacoplar por completo la representación visual en pantalla de la lógica de presentación y del modelo de datos subyacente.



a. El Modelo (Model)

Representa la entidad pura del dominio, es independiente de la interfaz gráfica de usuario (UI) o marco de trabajo móvil. En nuestro proyecto, la clase Post modela las publicaciones técnicas consumidas desde la API externa:

```
public class Post
{
    [JsonPropertyName("userId")]
    public int UserId { get; set; }

    [JsonPropertyName("id")]
    public int Id { get; set; }

    [JsonPropertyName("title")]
    public string Title { get; set; } = string.Empty;

    [JsonPropertyName("body")]
    public string Body { get; set; } = string.Empty;
}
```

Su única función es actuar como estructura de datos tipada. No posee lógica de validación visual ni notifica cambios a la pantalla de forma directa.

b. La Vista (View)

Compuesta por las páginas MainPage y DetallePage. Su rol se limita a declarar la distribución de los componentes visuales mediante XAML.

Para preservar la pureza del patrón, el archivo de código subyacente (code-behind, .xaml.cs) no ejecuta cálculos, peticiones de red ni almacenamiento de variables. Su única responsabilidad técnica es recibir el ViewModel mediante el contenedor de Inyección de Dependencias y asociarlo como contexto de enlace (BindingContext):

```
public partial class MainPage : ContentPage
{
    public MainPage(MainViewModel viewModel)
    {
        InitializeComponent();
        BindingContext = viewModel;
    }
}
```

Toda la interacción del usuario se resuelve declarativamente en el XAML mediante enlaces a propiedades y comandos:

```
<Button Text="Cargar Publicaciones"
        Command="{Binding CargarPostsCommand}" />
```

c. El ViewModel (ViewModel)

Actúa como intermediario reactivo entre la vista y la capa de servicios. Contiene el estado de la pantalla (EstaOcupado, MensajeEstado, ColorEstado, Posts), implementa la notificación de cambios mediante INotifyPropertyChanged y expone las acciones ejecutables del usuario a través de abstracciones de comandos (IAsyncRelayCommand).

Por ejemplo, en MainViewModel, el método que orquesta la carga de datos no manipula etiquetas visuales de forma imperativa; en su lugar, altera propiedades observables:

```
[RelayCommand(CanExecute = nameof(NoEstaOcupado))]  
private async Task CargarPostsAsync()  
{  
    EstaOcupado = true;  
    MensajeEstado = "Consultando servicio remoto...";  
    ColorEstado = Colors.DodgerBlue;  
  
    var resultado = await _postService.ObtenerPostsAsync();  
  
    // El ViewModel evalúa el resultado y muta su estado interno,  
    // notificando automáticamente a la UI a través del motor de  
    enlace de MAUI.  
    ...  
}
```

2. Data Binding y Comandos

Para evitar la duplicación de configuraciones y permitir el desarrollo en paralelo sin generar conflictos en archivos de escena, se construyó una jerarquía basada en herencia:

a. Verificación del BindingContext

Si el BindingContext de la página es null, el motor de XAML evalúa los enlaces de forma silenciosa y el comando jamás se ejecuta.

- i. **Diagnóstico en nuestro código:** Inspeccionar MainPage.xaml.cs. Comprobar que el constructor esté recibiendo la instancia de MainViewModel inyectada y que la asignación BindingContext = viewModel; se ejecute inmediatamente después de InitializeComponent().
- ii. **Prueba rápida:** Colocar un punto de interrupción (breakpoint) en el constructor de la vista para corroborar que viewModel no llegue nulo desde el contenedor de DI.

b. Coincidencia nominal del enlace y Source Generators

El enlace en XAML (Command="{Binding CargarPostsCommand}") busca una propiedad pública con ese nombre exacto.

- i. **Diagnóstico:** Como en el proyecto utilizamos [RelayCommand] sobre el método private async Task CargarPostsAsync(), el Source Generator crea por convención una propiedad pública denominada CargarPostsCommand (sustituyendo el sufijo Async por Command). Si en el XAML se hubiera tipiado Command="{Binding CargarPosts}" o Command="{Binding CargarPostsAsyncCommand}", el enlace fallaría de forma silenciosa.
- ii. **Herramienta:** Revisar la ventana Salida (Output) de Visual Studio filtrando por Depuración (Debug), donde el subsistema de enlace de XAML emite advertencias del estilo: Binding: 'CargarPosts' property not found on

'DevPostsApp.ViewModels.MainViewModel'

c. Evaluación de la guarda condicional (CanExecute)

Nuestro comando está acoplado declarativamente a la propiedad NoEstaOcupado:

```
[RelayCommand(CanExecute = nameof(NoEstaOcupado))]  
private async Task CargarPostsAsync()
```

- i. **Diagnóstico:** Si la bandera EstaOcupado quedó clavada en true (por ejemplo, debido a una excepción prematura que no pasó por el bloque finally), NoEstaOcupado evaluará en false. En consecuencia, el botón quedará inhabilitado a nivel de UI o el comando rechazará la invocación.
- ii. **Verificación:** Monitorear el valor en tiempo de ejecución de _estaOcupado y constatar que el atributo [NotifyCanExecuteChangedFor(nameof(CargarPostsCommand))] esté refrescando el estado del botón al finalizar la operación asíncrona.

d. Bloqueo del hilo de interfaz gráfica (UI Thread)

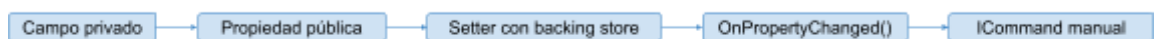
- i. **Diagnóstico:** Si la llamada al servicio de red se estuviera ejecutando de forma sincrónica forzada (e.g., _postService.ObtenerPostsAsync().Result o .Wait()), se produciría un bloqueo mutuo (deadlock) sobre el hilo principal de renderizado, congelando el despachador de eventos de la aplicación.
- ii. **Inspección de código:** Verificar que la invocación conserve estrictamente el modificador await sobre la tarea devuelta por el cliente de red.

3. Uso del CommunityToolkit.MVVM

En el marco del desarrollo de la solución, se optó por incorporar formalmente la biblioteca oficial CommunityToolkit.Mvvm.

La decisión se fundamentó tras evaluar el balance entre legibilidad, rendimiento y el código repetitivo (boilerplate) que exige la implementación manual de MVVM en C#.

Implementación Tradicional:



Con CommunityToolkit.Mvvm (Source Generators):

[ObservableProperty] private bool _estaOcupado; ————— Genera código reactivo en compilación (0 overhead)

a. Ventajas constatadas en nuestro proyecto

- i. **Reducción sustancial del código repetitivo:**
 En lugar de escribir manualmente campos de respaldo, validaciones de igualdad y métodos auxiliares de invocación de eventos (SetProperty / OnPropertyChanged), el atributo [ObservableProperty] permite que el compilador genere la propiedad pública reactiva en tiempo de compilación.

ii. **Generación declarativa de comandos ([RelayCommand]):**

Automatiza la instanciación de objetos Command en el constructor y el control de asincronía (IAsyncRelayCommand), gestionando de forma natural la reevaluación de la condición de ejecución mediante NotifyCanExecuteChangedFor.

iii. **Cero impacto en rendimiento en tiempo de ejecución:**

Al operar mediante Source Generators de Roslyn en lugar de introspección por reflexión (Reflection), todo el código de infraestructura se resuelve en tiempo de compilación, siendo completamente compatible con la compilación anticipada (AOT) y el recorte de ensamblados (trimming) requerido en dispositivos móviles.

b. Consideraciones y desventajas evaluadas

i. **Curva de comprensión para quien inicia:**

Requiere comprender que las clases deben declararse necesariamente con el modificador parcial, ya que el compilador complementa la definición de la clase en archivos físicos independientes generados dinámicamente ([.g.cs](#)).

ii. **Dependencia externa de paquete:**

Introduce una dependencia NuGet al proyecto, aunque al tratarse de un componente mantenido oficialmente por Microsoft y parte del ecosistema .NET MAUI estándar, el riesgo de obsolescencia o incompatibilidad queda minimizado.

4. Consumo de APIs y Estrategia de Manejo de Errores

El consumo de servicios web en entornos móviles presenta un desafío crítico: la inestabilidad inherente de las redes celulares y Wi-Fi exige distinguir con precisión quirúrgica una falla física de enlace frente a una respuesta HTTP anómala devuelta por el servidor.

a. Estrategia de Diseño: Patrón Envoltorio (Result Pattern)

En lugar de propagar excepciones crudas hacia la interfaz de usuario, se implementó una estructura genérica ResultadoOperacion<T> que encapsula el resultado de la comunicación:

```
public class ResultadoOperacion<T>
{
    public bool Exitoso { get; set; }
    public T? Datos { get; set; }
    public string Mensaje { get; set; } = string.Empty;
    public TipoError Error { get; set; } = TipoError.Ninguno;
}
```

A través de un catálogo tipado (TipoError), la aplicación categoriza la naturaleza del incidente: SinConexion, ErrorServidor, ErrorCliente o FormatoInvalido.

b. Diferenciación técnica de fallos en PostService

Dentro del servicio, el bloque de ejecución diferencia las fallas mediante captura jerárquica de excepciones y análisis del código de estado HTTP (Status Code):

```
try
{
    using var respuesta = await _httpClient.GetAsync(ApiUrl);

    if (!respuesta.IsSuccessStatusCode)
    {
        return ClasificarErrorHttp(respuesta.StatusCode);
    }

    var contenidoJson = await respuesta.Content.ReadAsStringAsync();

    var opciones = new JsonSerializerOptions
    {
        PropertyNameCaseInsensitive = true
    };

    var posts = JsonSerializer.Deserialize<List<Post>>(contenidoJson,
opciones);

    if (posts == null || posts.Count == 0)
    {
        return ResultadoOperacion<List<Post>>.Fallo(
            "No se encontraron publicaciones disponibles.",
            TipoError.ErrorServidor);
    }

    return ResultadoOperacion<List<Post>>.Ok(
        posts,
        $"Se cargaron {posts.Count} publicaciones correctamente.");
}
catch (TaskCanceledException)
{
    return ResultadoOperacion<List<Post>>.Fallo(
        "La solicitud tardó demasiado tiempo en responder (Timeout).  
Verifique su conexión.",
        TipoError.SinConexion);
}
catch (HttpRequestException ex)
{
    return ResultadoOperacion<List<Post>>.Fallo(
        $"Fallo de red: No fue posible conectar con el servidor
```

```
({ex.Message}).",  
        TipoError.SinConexion);  
}  
catch (JsonException)  
{  
    return ResultadoOperacion<List<Post>>.Fallo(  
        "Error de formato: Los datos recibidos no tienen la  
estructura esperada.",  
        TipoError.FormatoInvalido);  
}
```

El método privado ClasificarErrorHttp desglosa el rango del código de estado:

- i. **Códigos 4xx (Errores de Cliente):** Distingue situaciones como 404 Not Found (recurso inexistente) o 401/403 (autorización), catalogándolos como TipoError.ErrorCliente.
- ii. **Códigos 5xx (Errores de Servidor):** Identifica anomalías internas del backend (500 Internal Server Error, 503 Service Unavailable), clasificándolos como TipoError.ErrorServidor.

c. Justificación de la solución

Esta arquitectura ofrece dos ventajas fundamentales:

- i. **Manejo controlado de errores de la petición:**
El servicio transforma los errores conocidos de red, timeout, HTTP y formato en resultados clasificados. El ViewModel presenta un mensaje contextual en lugar de dejar que esos fallos esperables interrumpan el flujo normal de carga
- ii. **Retroalimentación visual contextual:**
El ViewModel mapea el enum TipoError a colores y mensajes específicos para el usuario (e.g., color carmesí para falta de internet, naranja para fallas de servidor), permitiéndole al usuario saber si debe revisar su señal de datos o si se trata de un mantenimiento de la plataforma externa.

5. Navegación en MAUI y Escalabilidad Estructural

a. Resolución de la Navegación en el Proyecto

Para la aplicación se utilizó la arquitectura declarativa de .NET MAUI Shell, descartando la navegación jerárquica manual clásica basada en NavigationPage.

La navegación hacia la pantalla de detalle se estructuró a partir de dos decisiones técnicas clave:

i. Registro contextual de rutas:

En AppShell.xaml.cs se registró la vista de destino desacoplándola de la navegación estática:

```
Routing.RegisterRoute(nameof(DetallePage), typeof(DetallePage));
```

ii. Transferencia de entidades complejas sin fragmentación:

En lugar de serializar primitivos sobre la URL (e.g., \$"DetallePage?id={post.Id}&title={post.Title}"), lo cual expone la navegación a errores de escape de caracteres, truncamientos y obliga a reconsultar la red, se utilizó ShellNavigationQueryParameters:

```
var parametros = new ShellNavigationQueryParameters
{
    { "PostSeleccionado", post }
};
await Shell.Current.GoToAsync("DetallePage", parametros);
```

iii. Recepción limpia vía IQueryAttributable:

DetalleViewModel implementa la interfaz IQueryAttributable, abstrayendo la llegada de datos en el método ApplyQueryAttributes. Esto garantiza una vinculación tipada, segura y compatible con el ciclo de vida del contenedor de inyección de dependencias. Para el regreso al listado, se invoca la ruta relativa .. (Shell.Current.GoToAsync("..")), conservando la memoria previa del listado y la posición del scroll intactos.

b. Alternativa ante un proyecto con múltiples secciones

Si la aplicación requiriera evolucionar hacia un producto integral con múltiples módulos (por ejemplo: Listado de Posts, Categorías/Etiquetas, Publicaciones Guardadas en Favoritos y Perfil del Desarrollador), la alternativa arquitectónica ideal seguiría siendo .NET MAUI Shell, pero estructurada bajo un esquema de Navegación por Pestañas Inferiores (TabBar / Tab) y Menú Lateral (FlyoutItem).

```
<Shell xmlns="http://schemas.microsoft.com/dotnet/2021/maui" ...>
  <TabBar>
    <Tab Title="Explorar" Icon="feed_icon.png">
      <ShellContent Route="MainPage"
ContentTemplate="{DataTemplate views:MainPage}" />
    </Tab>
    <Tab Title="Favoritos" Icon="star_icon.png">
      <ShellContent Route="FavoritosPage"
ContentTemplate="{DataTemplate views:FavoritosPage}" />
    </Tab>
    <Tab Title="Perfil" Icon="user_icon.png">
      <ShellContent Route="PerfilPage"
ContentTemplate="{DataTemplate views:PerfilPage}" />
    </Tab>
  </TabBar>
```

```
</Shell>
```

6. Justificación de esta elección frente a `NavigationPage` o

`TabbedPage`

- **Gestión centralizada del árbol de navegación:**

Shell unifica en un único documento XAML la definición estructural de la aplicación, soportando de forma nativa jerarquías visuales mixtas (pestañas superiores, barras inferiores y menú hamburguesa lateral) sin requerir anidamientos complejos de contenedores.
- **Carga perezosa de vistas (`DataTemplate`):**

Mediante la propiedad `ContentTemplate="{DataTemplate views:Pagina}"`, Shell retrasa la instanciación de las pantallas secundarias hasta el momento exacto en que el usuario presiona su pestaña, optimizando el consumo de memoria RAM y acelerando el tiempo de arranque (startup) de la aplicación móvil.
- **Compatibilidad plena con Deep Linking:**

Al poseer un enrutador basado enteramente en URIs relativas y absolutas, el sistema facilita la navegación directa hacia pantallas internas desde notificaciones push o enlaces externos.